



Basic Setup

If your app already has an older version of Sparkle or you wish to migrate to Sparkle 2, please see [upgrading from previous versions](#).

If you want to sandbox an existing application using Sparkle, please jump to the [sandboxing guide](#).

1. Add the Sparkle framework to your project

If you use [Swift Package Manager](#):

- In your Xcode project: File › Add Packages...
- Enter <https://github.com/sparkle-project/Sparkle> as the package repository URL
- Choose the Package Options. The default options will let Xcode automatically update versions of Sparkle 2.

From Xcode's project navigator, if you right click and show the [Sparkle](#) package in Finder, you will find Sparkle's tools to generate and sign updates in [../artifacts/sparkle/Sparkle/bin/](#) (in Finder you may need to go up one folder from [checkouts](#) via [Go](#) › [Enclosing Folder](#)).

If you use [Carthage](#):

- Add `binary "https://sparkle-project.org/Carthage/Sparkle.json"` to your Cartfile.
- Run `carthage update`
- Link the Sparkle framework to your app target:
 - Drag the built `Carthage/Build/Mac/Sparkle.framework` into your Xcode project.
 - Make sure the box is checked for your app's target in the sheet's Add to targets list.
- Make sure the framework is copied into your app bundle:
 - Click on your project in the Project Navigator.
 - Click your target in the project editor.
 - Click on the General tab.
 - In Frameworks, Libraries, and Embedded Content section, change Sparkle.framework to Embed & Sign.

Sparkle's tools to generate and sign updates are not included from Carthage and need to be grabbed from [our latest release](#).

Sparkle only supports using a `binary` origin with Carthage because Carthage strips necessary code signing information when building the project from source.

If you use [CocoaPods \(deprecated\)](#):

- Add `pod 'Sparkle'` to your Podfile.
- Add or uncomment `use_frameworks!` in your Podfile.

If you want to add Sparkle manually:

- Get the [latest version](#) of Sparkle.
- Link the Sparkle framework to your app target:
 - Drag `Sparkle.framework` into your Xcode project.
 - Be sure to check the "Copy items into the destination group's folder" box in the sheet that appears.
 - Make sure the box is checked for your app's target in the sheet's Add to targets list.
- Make sure the framework is copied into your app bundle:
 - Click on your project in the Project Navigator.
 - Click your target in the project editor.
 - Click on the General tab.
 - In Frameworks, Libraries, and Embedded Content section, change Sparkle.framework to Embed & Sign.
- In Build Settings tab set "Runpath Search Paths" to `@loader_path/../Frameworks` (for non-Xcode projects add the flags `-Wl,-rpath,@loader_path/../Frameworks`). By default, recent versions of Xcode set this to `@executable_path/../Frameworks` which is already sufficient for regular applications.
- If you have your own process for copying/packaging your app make sure it preserves symlinks and executable permissions!

If you enable Library Validation, which is part of the Hardened Runtime and required for notarization, you will also need to either sign your application with an [Apple Development](#) certificate for development (requires being in Apple's developer program), or disable library validation for Debug configurations only. Otherwise, the system may not let your application load Sparkle if you attempt to sign to run locally via an ad-hoc signature. This is not an issue for distribution when you sign your application with a Developer ID certificate.

If your application is sandboxed, please also follow the [sandboxing guide](#). Otherwise, you may optionally be interested in [removing Sparkle's XPC Services](#) to save space.

[Pre-releases](#) when available are published on GitHub. They are also available in Swift Package Manager, Carthage, and CocoaPods too by specifying the pre-release version in your project's manifest.

A more nightly build from our repository can be downloaded from our [GitHub Actions page](#) by selecting a recent workflow commit and downloading the `Sparkle-distribution*.tar.xz` artifact. Alternatively, you may clone Sparkle's repository, run `make release`, and extract the binaries in the resulting `Sparkle-*.tar.xz` archive.

2. Set up a Sparkle updater object

These instructions are for regular .app bundles in Cocoa. If you want to use Sparkle from other UI toolkits such as SwiftUI or want to instantiate the updater yourself, please visit [our programmatic setup](#). If you want to update a non-app bundle, such as a Preference Pane or a plug-in, follow [step 2 for non-app bundles](#).

- Open up your MainMenu.xib.
- Choose View › Show Library...

- Type “Object” in the search field under the object library and drag an Object into the left sidebar of the document editor.
- Select the Object that was just added.
- Choose View › Inspectors › Identity.
- Type `SPUStandardUpdaterController` in the Class box of the Custom Class section in the inspector.
- If you’d like, make a “Check for Updates...” menu item in the application menu; set its target to the `SPUStandardUpdaterController` instance and its action to `checkForUpdates:`.

If you are using Sparkle 1, you will need to use `SUUpdater` instead of `SPUStandardUpdaterController` in the above steps. In Sparkle 2, `SUUpdater` is a deprecated stub. While it is still functional for transitional purposes, new applications will want to migrate to `SPUStandardUpdaterController`.

That’s it. No other API calls are required to start the updater and have it manage checking for updates automatically. If you intend to pursue additional updater APIs, please first check [API Expectations](#) from our programmatic setup.

3. Segue for security concerns

Because Sparkle is installing executable code to your users’ systems, you must be very careful about security. To let Sparkle know that a downloaded update is not corrupted and came from you (instead of a malicious attacker), we recommend:

- Serve updates over HTTPS and comply with Apple’s [App Transport Security requirements](#).
- Notarize and code sign the application via Apple’s Developer ID program (if possible). To diagnose basic code signing issues, you can run `codesign --deep --verify <path-to-app>`.
- Sign the published update archive (dmg, zip, etc), [binary delta updates](#), and [installer packages](#) with Sparkle’s EdDSA (ed25519) signature.

Please ensure your signing keys are kept safe and cannot be stolen if your web server is compromised. One way to ensure this for example is not having your signing keys accessible from the machine that is hosting your product.

EdDSA (ed25519) signatures

To prepare signing with EdDSA signatures:

Run `./bin/generate_keys` tool (from the Sparkle distribution root). This needs to be done only once. This tool will do two things:

- It will generate a private key and save it in your login Keychain on your Mac. You don’t need to do anything with it, but do keep it safe. See further notes below if you happen to lose your private key.
- It will print your public key to embed into applications. Copy that key (it’s a base64-encoded string). You can run `./bin/generate_keys` again to see your public key at any time.

Then add your public key to your app’s `Info.plist` as a `SUPublicEDKey` property. Note that for new projects created with Xcode 12 or later, this file may be in the Info tab under your target settings.

Here is an example run of `./bin/generate_keys`:

```
A key has been generated and saved in your keychain. Add the `SUPublicEDKey` key to
the Info.plist of each app for which you intend to use Sparkle for distributing
updates. It should appear like this:

<key>SUPublicEDKey</key>
<string>pfIShU4dEXqPd50bYNfDBiQWcXozk7estwzTnF9BamQ=</string>
```

Be sure to keep your keys safe and not lose them (they will be erased if your keychain or system is erased). You can use the `-x private-key-file` and `-f private-key-file` options to export and import the keys respectively when transferring keys to another Mac. Otherwise we recommend keeping the keys inside your Mac’s keychain. If your keys are lost however, you can still sign new updates for Developer ID signed applications through [key rotation](#).

Please visit [Migrating to EdDSA from DSA](#) if you are still providing DSA signatures so you can learn how to stop supporting them.

Signing feeds (optional)

For greater security, you can opt into signing your update feed and release notes by enabling `SURequireSignedFeed`. This validates the information presented to the user. For example, with a signed feed, an attacker that compromises an app’s update server will not be able to inform and trick existing users to update from another location.

`SUVerifyUpdateBeforeExtraction` must also be enabled to use signed feeds. This option enforces validation of update archives before extracting them.

With these options, more responsibility is on the developer to keep their signing keys safe and ensure their feeds and release notes stay correctly signed (modifications to these files will require re-signing them). For signed feeds, the `SUSignedFeedFailureExpirationInterval` option allows a limited fallback through [key rotation](#) in case a feed hasn’t been successfully validated for a lengthy period of time. Note when feed signing failures expire, Sparkle strips out information like release notes, links, and certain elements.

Signed feeds are validated as of Sparkle 2.9 (beta).

Rotating signing keys

For regular application updates (not installer package updates), if you both code-sign your application with Apple’s Developer ID program and include a public EdDSA key for signing your update archive, Sparkle allows rotating keys by issuing a new update that changes either your Apple code signing certificate or your EdDSA keys (but not both). For applications that opt into enabling `SUVerifyUpdateBeforeExtraction`, changing your EdDSA keys can only be done if the update archive is a Developer ID code signed disk image (dmg).

We recommend rotating keys only when necessary like if you need to change your Developer ID certificate, lose access to your EdDSA private key, or need to change (Ed)DSA keys due to [migrating away from DSA](#).

4. Distributing your App

We recommend building your distributable app in Xcode by creating a Product › Archive and Distribute App choosing Developer ID method of distribution. Using Xcode’s Archive Organizer will ensure Sparkle’s helper tools are code signed properly for distribution. In automated environments, this process can be done using `xcodebuild archive` and `xcodebuild -exportArchive`.

If you distribute your app on your website as a [notarized and Developer ID signed disk image](#) (recommended), add an `/Applications` symlink in your disk image to encourage users to copy the app out of it.

If you distribute your app on your website as a zip or a tar archive, avoid placing anything but your app inside the archive so you can minimize [app translocation](#) issues.

Note by default Sparkle will not notify your user if an update cannot be performed, like if the app is running from a read-only mount or being impacted by app translocation.

Sparkle supports updating from dmg, zip archives, tarballs, Apple Archives (as of Sparkle 2.7 / macOS 10.15), and [installer packages](#), so you can generally reuse the same archive for distribution of your app on your website as well as Sparkle updates.

5. Publish your appcast

Sparkle uses appcasts to get information about software updates. An appcast is an RSS feed with some extra information for Sparkle's purposes.

- Add a `SUFeedURL` property to your `Info.plist`; set its value to a URL where your appcast will be hosted, e.g. `https://yourcompany.example.com/appcast.xml`.
- Note that your app bundle must have an incrementing and [properly formatted](#) `CFBundleVersion` key in your `Info.plist`. Sparkle uses this to compare and determine the latest version of your bundle.

If you update regular app bundles and you have set up EdDSA signatures, you can use a tool to generate appcasts automatically:

1. Build your app and compress it (e.g. in a dmg/zip/tar.xz/aar archive), and put the archive in a new folder. This folder will be used to store all your future updates.
2. Run `generate_appcast` tool from Sparkle's distribution archive specifying the path to the folder with update archives. Allow it to access the Keychain if it asks for it (it's needed to generate signatures in the appcast).

```
./bin/generate_appcast /path/to/your/updates_folder/
```

3. The tool will generate the appcast file (using filename from `SUFeedURL`) and also `*.delta` update files for faster incremental updates. Upload your archives, the delta updates, and the appcast to your server.

When generating the appcast, if an `.html` file exists with the same name as the archive, then it will be added as the `releaseNotesLink`. As of Sparkle 2.9 (beta) and macOS 12, basic markdown support using a `.md` file has also been added. Run `generate_appcast -h` for a full overview and list of supported options.

If your app opts into [SURequireSignedFeed](#), `generate_appcast` will also sign your appcast and release note files. If you make further modifications to your appcast or release notes, you will need to re-run `generate_appcast` to generate updated signatures.

You can also create the appcast file manually (not recommended):

- Make a copy of the sample appcast included in the Sparkle distribution.
- Read the sample appcast to familiarize yourself with the format, then edit out all the items and add one for the new version of your app by following the instructions at [Publishing an update](#).

6. Test Sparkle out

- Use an *old* version of your app
 - Decreasing the `CFBundleVersion` of the latest development build of your app temporarily is useful for quickly testing the latest version of Sparkle framework.
 - Creating a stub notarized build of your app that has a very high `CFBundleVersion` hosted in an [alternate feed](#) is useful for testing the latest version of Sparkle framework in a production setting. Some code signing policies may otherwise not take effect and be testable when updating from/to non-notarized builds.
 - A genuine older and newer version of the app will be required to test [delta updates](#), because Sparkle will ignore the delta update if the app doesn't match update's checksum.
- Run the app, then quit. By default, Sparkle doesn't ask the user's permission for checking updates until the *second* launch, in order to make your users' first-launch impression cleaner.
- Run the app again. The update process should proceed as expected. Note by default, Sparkle checks for updates in the background once every 24 hours. To test automatic update checks immediately, run `defaults delete my-bundle-id SULastCheckTime` to clear the last update check time before launching the app. Alternatively, initiate a manual update check from the app's menu bar.

The update process will be logged to `Console.app`. If anything goes wrong, you should find detailed explanation in the log.

Make sure to also keep Sparkle's debug symbols files (`.dSYM`) around as they will be useful for symbolizing crash logs if something were to go wrong.

Next steps

That's it! You're done! You don't have to do any more. But you might want to:

- [Read more on publishing an update](#)
- [Customize Sparkle's settings and behavior](#) for your product.
- [Add update settings](#) to your settings panel.
- [Add binary delta updates](#) to your application.
- [Add gentle update reminders](#) for your application.
- [Add a custom user interface](#) for your application's updater.
- [Learn about gathering anonymous statistics about your users' systems](#).
- [Review Sparkle 2's API Reference](#)